

ARCHITECTURE.md — CleanPix (AI Background Remover)

Derived from PRD.md and DESIGN.md. No new features introduced; MVP scope unchanged (guest-first, no login required for V1; accounts/history/projects are the natural extension point for the stated “team workspace” future-scope item, and are marked **optional/Phase 2** throughout this doc).

1. Architecture Overview

CleanPix is a **client-heavy SaaS app** with a thin backend that exists mainly to broker AI processing and file storage.

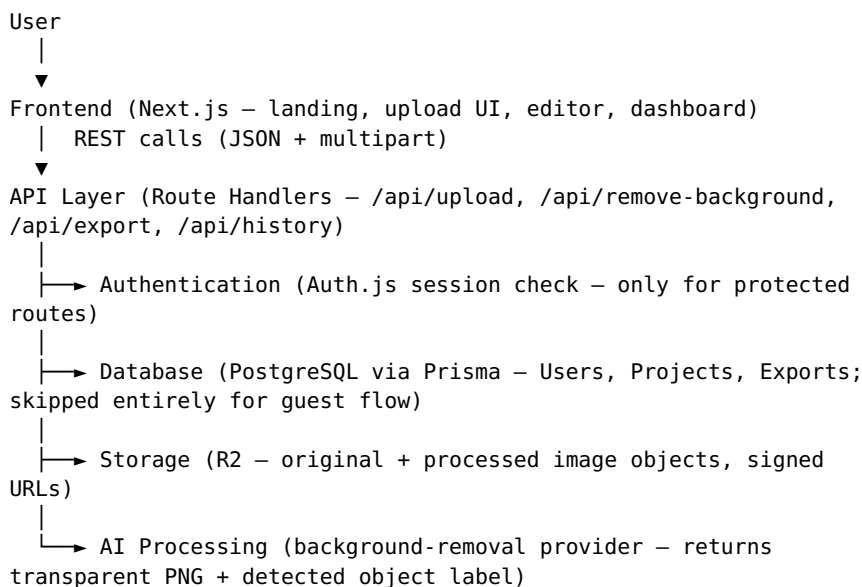
- **Frontend** — handles upload UI, before/after preview, slider, social-kit crop selection, and download. Talks to the backend only via a small REST API.
- **Backend (API layer)** — stateless API that validates uploads, calls the AI service, and returns processed image URLs. No business logic beyond orchestration.
- **Database** — stores only what’s needed for optional accounts and history (Phase 2). MVP can run with **zero persistent DB writes** for guest users (job state lives in memory/cache + storage).
- **Authentication** — off by default (guest mode = MVP). When enabled, it gates only “Save Project” / “History” / “Dashboard” — never the core remove-background flow.
- **Storage** — temporary object storage for originals and processed outputs, auto-expiring for guests.
- **AI Processing** — a dedicated background-removal service (managed API or self-hosted model) invoked synchronously/async by the backend; returns a transparent PNG + metadata (detected object type) used to drive smart background suggestions.

2. Recommended Tech Stack

Layer	Choice	Why
Frontend framework	Next.js 14 (App Router)	SSR for fast marketing/SEO pages + client components for interactive editor; one codebase for landing + dashboard
Language	TypeScript	Type-safe contracts between frontend/API, fewer runtime bugs
Styling	Tailwind CSS + CSS variables for DESIGN.md tokens	Matches dark/glass/gradient system exactly via custom theme config
Backend API	Next.js Route Handlers (or NestJS if the API grows beyond a few routes)	Colocated with frontend for MVP speed; NestJS as a clean cut-over if the team scales
		Relational integrity for

Database	PostgreSQL	Users/Projects/Exports; easy Phase-2 addition
ORM	Prisma	Type-safe schema, migrations, pairs naturally with TypeScript
Auth provider	Auth.js (NextAuth) — Google OAuth + Email/Magic Link	Matches PRD's "optional login," no password infra to manage
Image storage	Cloudflare R2 (S3- compatible)	Cheap egress for image-heavy traffic, signed URLs, lifecycle rules for auto-delete
AI background removal	Managed API (e.g., remove.bg-style provider) for MVP; swappable for a self- hosted rembg/U ² -Net service later	Fastest path to MVP quality without owning GPU infra; interface is abstracted so the provider can be swapped
Frontend deployment	Vercel	Native Next.js support, edge caching, zero- config previews
Backend deployment	Vercel (co-located routes) → migrate hot paths to a dedicated Node service (Railway/Fly.io) if processing volume needs a worker queue	Start simple, scale the bottleneck only
Analytics	PostHog (self-hosted or cloud)	Product analytics + funnels (upload → download conversion) without vendor lock-in

3. System Architecture



Layer responsibilities - Frontend: rendering, client-side validation, optimistic UI, slider/preview state. - **API Layer:** request validation, auth gate, orchestration between storage and AI provider, response shaping. - **Authentication:** identifies a user *if logged in*; guests pass through untouched. - **Database:** durable record of accounts/projects/exports only — never in the guest hot path. - **Storage:** source of truth for image bytes; DB only stores references (URLs/keys). - **AI Processing:** single-purpose external call; stateless from the app’s perspective.

4. User Flows

Guest User Flow (MVP default) Landing Page → Upload → Background Removal → Preview (before/after slider) → Download - No account created. Job exists only for the session; storage objects auto-expire (e.g., 24h TTL).

Logged-In User Flow (Phase 2, optional) Login → Dashboard → Upload → Remove Background → Save Project → Download - Identical processing pipeline; the only addition is a DB write (Project row) after a successful export, and a History/Dashboard view reading from it.

5. Frontend Architecture

Pages | Route | Purpose | |—|—| | / | Landing (hero, features, pricing, FAQ per DESIGN.md homepage structure) | | /login | Sign-in (optional, Phase 2) | | /dashboard | Logged-in home: credits, recent edits, quick actions (Phase 2) | | /editor/[jobId] | Upload → processing → preview/slider → download (used by both guest and logged-in flows) | | /history | Past projects list (Phase 2, requires auth) |

Component structure — feature-first, e.g.:

```
components/
├─ ui/           # buttons, inputs, cards – DESIGN.md primitives
├─ upload/       # Dropzone, FilePreview, ProgressState
├─ editor/       # BeforeAfterSlider, BackgroundSuggestions,
SocialKitExporter
├─ dashboard/    # CreditsCard, RecentEditsGrid, QuickActions
(Phase 2)
└─ layout/       # Navbar, Sidebar, Footer
```

State management: local component state + React Query (TanStack Query) for server state (upload/job status, history). No global store needed at MVP size; adopt Zustand only if cross-page editor state emerges.

Form handling: React Hook Form + Zod schemas shared between client validation and API route validation (single source of truth for file-type/size rules).

6. Backend Architecture

Service	Responsibility
Authentication Service	Session issuance/validation via Auth.js; no-op for guest requests Validates file (type/size), writes original to storage, creates an

Upload Service	ephemeral job record (cache, not DB)
AI Processing Service	Calls the background-removal provider with the stored original, writes the processed result back to storage, returns detected-object metadata Generates social-kit crops (1:1, 9:16, 16:9, LinkedIn) from the processed PNG and returns signed download URLs
Export Service	

Interaction: Upload Service → AI Processing Service → Export Service, chained per job; each step is independently retryable and idempotent by jobId.

7. Database Design

(Used only for optional accounts/history — MVP guest flow does not touch these tables.)

User	
└ id	PK, uuid
└ email	unique
└ name	
└ authProvider	(google email)
└ credits	int, default free-tier value
└ createdAt	
Project	
└ id	PK, uuid
└ userId	FK → User.id
└ originalUrl	
└ processedUrl	
└ detectedObject	(person product pet vehicle other)
└ status	(pending processing done failed)
└ createdAt	
Export	
└ id	PK, uuid
└ projectId	FK → Project.id
└ format	(instagram_post instagram_story youtube_thumb linkedin_profile original_png)
└ url	
└ createdAt	

Relationships: User 1-* Project, Project 1-* Export. All FKs cascade on delete.

8. API Design

Route	Method	Request	Response
/api/upload	POST	multipart/form-data (file)	{ jobId, originalUrl }
/api/remove-background	POST	{ jobId }	{ jobId, processedUrl, detectedObject, status }
/api/project/:id	GET	— (auth	{ project,

		required)	exports[] }
/api/history	GET	— (auth required)	{ projects: [...] }
/api/export	POST	{ jobId, format }	{ url, format }

Error handling (uniform shape):

```
{ "error": { "code": "FILE_TOO_LARGE", "message": "Max size is 10MB." } }
```

Standard codes: INVALID_FILE_TYPE, FILE_TOO_LARGE, AI_PROCESSING_FAILED, UNAUTHORIZED, NOT_FOUND, RATE_LIMITED.

9. AI Processing Pipeline

Upload → Validation (type/size) → Temporary Storage
 → Background Removal (AI provider call)
 → Optimization (compress/resize output)
 → Export Generation (social-kit crops, on demand)
 → Response (URLs + metadata)

Error scenarios: invalid file type/size (reject pre-upload), AI provider timeout/failure (retry once, then surface AI_PROCESSING_FAILED with a “try again” CTA), storage write failure (rollback job, no partial state exposed to user).

10. Folder Structure

```
src/
├─ app/           # Next.js routes (pages + API route handlers)
├─ components/    # Reusable UI (feature-first, see §5)
├─ features/      # Editor, dashboard, upload – feature-scoped
                    logic + local components
├─ lib/           # Auth.js config, Prisma client, storage client,
                    AI provider client
├─ hooks/         # useUpload, useJobStatus, useExport (React
                    Query wrappers)
├─ types/         # Shared TS types/Zod schemas (mirrors API
                    contracts)
├─ services/      # Thin server-side modules called by route
                    handlers (upload, ai, export)
├─ utils/         # Formatting, file validation helpers, constants
└─ styles/        # Tailwind config + DESIGN.md tokens (colors,
                    radii, shadows as CSS vars)
```

11. Authentication Architecture

- **Guest Mode** (default): no session; every route required for the core flow (/api/upload, /api/remove-background, /api/export) works unauthenticated.
- **Optional Google Login** and **Optional Email Login**: via Auth.js; purely additive.
- **Session management**: JWT session cookie (Auth.js default), no server-side session store needed at this scale.
- **Protected routes**: /dashboard, /history, /api/project/:id, /api/history check session; redirect to /login (pages) or 401 (API) if absent.
- **Dashboard access**: only reachable once authenticated; guests are

routed straight to /editor/[jobId] and never see a login wall in the core flow.

12. Security Considerations

- **File validation:** strict MIME-type allowlist (JPG/JPEG/PNG), max 10MB, magic-byte check server-side (never trust client-reported type).
- **Rate limiting:** per-IP limit on /api/upload and /api/remove-background (e.g., token bucket via middleware) to prevent AI-cost abuse from guests.
- **Image cleanup:** guest job objects auto-expire via storage lifecycle rules (24h); logged-in users’ data persists until deleted by them.
- **Authentication security:** HttpOnly, Secure, SameSite cookies; OAuth over Auth.js (no custom password storage in MVP).
- **API protection:** input validation via shared Zod schemas, CORS locked to the app’s own origin, signed/short-lived URLs for storage access.

13. Performance Strategy

- **Image optimization:** compress/resize processed PNGs server-side before storage; serve via next/image where displayed in-app.
- **Caching:** CDN-cache static/marketing pages; cache AI provider responses by content hash to avoid reprocessing identical uploads.
- **Lazy loading:** dashboard grids and history lists paginate/lazy-load; below-the-fold landing sections code-split.
- **CDN usage:** R2 fronted by Cloudflare CDN for all image delivery; Vercel edge network for static assets.
- **Request optimization:** debounce/batch client polling of job status; prefer short-poll with backoff over websockets at MVP scale.

14. Deployment Architecture

- **Frontend:** Vercel, connected to main branch, preview deployments per PR.
- **Backend:** co-located Vercel Route Handlers for MVP; environment variables manage provider keys and DB URL.
- **Database hosting:** managed Postgres (e.g., Neon or Supabase) — serverless-friendly connection pooling for Vercel’s function model.
- **Image storage:** Cloudflare R2 bucket, separate buckets/prefixes for originals/ and processed/.
- **Environment variables:** DATABASE_URL, AI_PROVIDER_API_KEY, R2_ACCESS_KEY/R2_SECRET, AUTH_SECRET, GOOGLE_CLIENT_ID/SECRET.

Flow: push → Vercel build → migrations run (prisma migrate deploy) → deploy → smoke test upload/remove-background/export endpoints.

15. Architecture Decisions & Trade-offs

Decision	Reasoning	Trade-off
Guest-first, DB-optional	Matches PRD’s “no login required in V1”; fastest time-to-value	Phase 2 auth/history retrofits require the DB path to be added carefully without

Next.js for frontend + API	One deploy target, fast MVP velocity, matches DESIGN.md's SSR-friendly marketing needs	breaking guest flow Tightly couples frontend/backend deploys; may need to split out backend if traffic/processing grows
Managed AI provider over self-hosted model	No GPU ops burden, ships MVP faster	Ongoing per-image cost, less control over model quality/latency
PostgreSQL + Prisma	Strong relational fit for Users/Projects/Exports, mature tooling	Slight overhead vs. NoSQL for a schema this simple — acceptable given future feature growth (teams, workspace)
Cloudflare R2 for storage	Cheap egress for image-heavy app	Slightly less ecosystem tooling than AWS S3
Auth.js (not custom auth)	Optional login must stay lightweight and not gate MVP flow	Less flexibility than a fully custom auth service if enterprise SSO is needed later
